When Evaluation Prevents Learning: Functional Accumulation in Self-Evolving Agents

Zonghe Wu[a,b,*]

[a]School of Architecture, Planning and Environmental Policy, University College Dublin, Belfield, Dublin 4, Ireland
[b]Chang'an-Dublin International College of Transportation, Chang'an University, Xi'an 710018, China

Abstract. In self-evolving AI agents, automated evaluators such as test assertions control what the agent learns. Across 2,468 executions, we show that flawed evaluators prevent the agent from correcting its own errors. An agent that omitted a required HR deferral passed all nine assertions and received no editing opportunity. In a sales task, outdated assertions pushed the agent to adopt an older announcement format that violates the latest business instruction. We prove that when assertions conflict with business requirements, the learning rule excludes the correct fix from all candidates. Beyond evaluation, we trace runtime breaks: an inherited component logged 27,500 invocations with zero effects, and a context-reorganization treatment failed to improve cross-task transfer. Feeding the agent direct traces from prior attempts, however, raised same-task success from 41/72 to 49/72. Misaligned evaluators miss the agent's errors and write their own into persistent memory.

1 Introduction

An employee on sabbatical requires a deferral record and must remain unassigned to courses. An automation agent omits the record, passes every assertion, and receives no opportunity to repair the omission. In a sales task, the agent follows the latest announcement instruction and fails its assertions. The editor adopts the older format rewarded by those assertions. Every legal edited candidate then passes the evaluator and violates the latest instruction. Evaluation determines which business failure becomes a learning object and supplies the interpretation written into experience.

Persistent agents learn through artifacts that outlive an execution. Reflections preserve lessons from failure [10]; Agentic Context Engineering (ACE) curates playbooks [16]; executable skills make experience callable [12]. Recent studies follow acquisition and retention across artifact revisions [11, 5]. This raises a question from biological evolvability: how does inherited structure produce functional variation that selection can retain? Kirschner and Gerhart place the generation of heritable, selectable variation at the center of evolvability [3]. For an agent, the evidence is a changed artifact that produces valid business behavior on later applicable opportunities.

We study this construction process in GNOSIS, a system for persistent repair around a fixed language model. A business obligation identifies the required action and affected object under the task's authority. We follow it from feedback through the editor's rationale and changed artifact to subsequent execution. Later applicable uses establish retention; new tasks test transfer. Figure 1 shows how evaluation acts along this path.

Evaluation controls both the production of variants and their later acceptance. A false pass removes a failure from editing. A false rejection can teach a correction that contradicts the authorized task. Our principal contribution traces these decisions from the score and eligibility branch to the changed artifact and deployed action. A conditional exclusion result states when the learning rule removes a necessary repair from consideration.

The evidence combines 2,006 historical executions with a frozen follow-up of 462 executions on six sources and 24 previously unexposed transfer tasks. Reconstruction joins 599 proposal records to version contents and follows 65 edits. Offline replay checks all 17,141 follow-up actions and 63 historical executions containing component effects. Complete business review supplies 11,554 obligation judgments. A 240-execution intervention within the historical population tests whether late access to knowledge explains global validation dependencies.

The runtime results locate failures after construction. An inherited component remains inactive despite repeated invocation, and fewer tool failures coexist with unchanged strict success. Direct task evidence improves the next attempt; the feedback organization intervention misses its transfer target. Source and target materials identify the authority conditions a reusable decision must preserve.

2 From inherited structure to accumulated function

2.1 An obligation supplies the functional identity

Schrödinger's account of living organization concerns its maintenance through exchange with the environment [8]; evolvability concerns the functional variants available to selection [3]. For an agent, these relations connect an obligation to an inherited procedure and that procedure to valid action. Runtime expression and task authority determine its function on each opportunity.

We identify a business obligation by the required action or value, its affected object, and the conditions authorizing it. Function is observed through satisfaction of that obligation in application state or recorded actions. Functional accumulation requires its renewed satisfaction through inherited artifacts on later applicable opportunities. A source correction establishes behavior on the source task; retention follows subsequent versions under the same obligation; transfer tests use in new tasks whose objects and authority must be established again.

The observation unit is one obligation exposed in one edit package, paired with the resulting candidate and its successor executions. The

* Phone: 13646028118; e-mail: zonghe.wu@ucdconnect.ie.

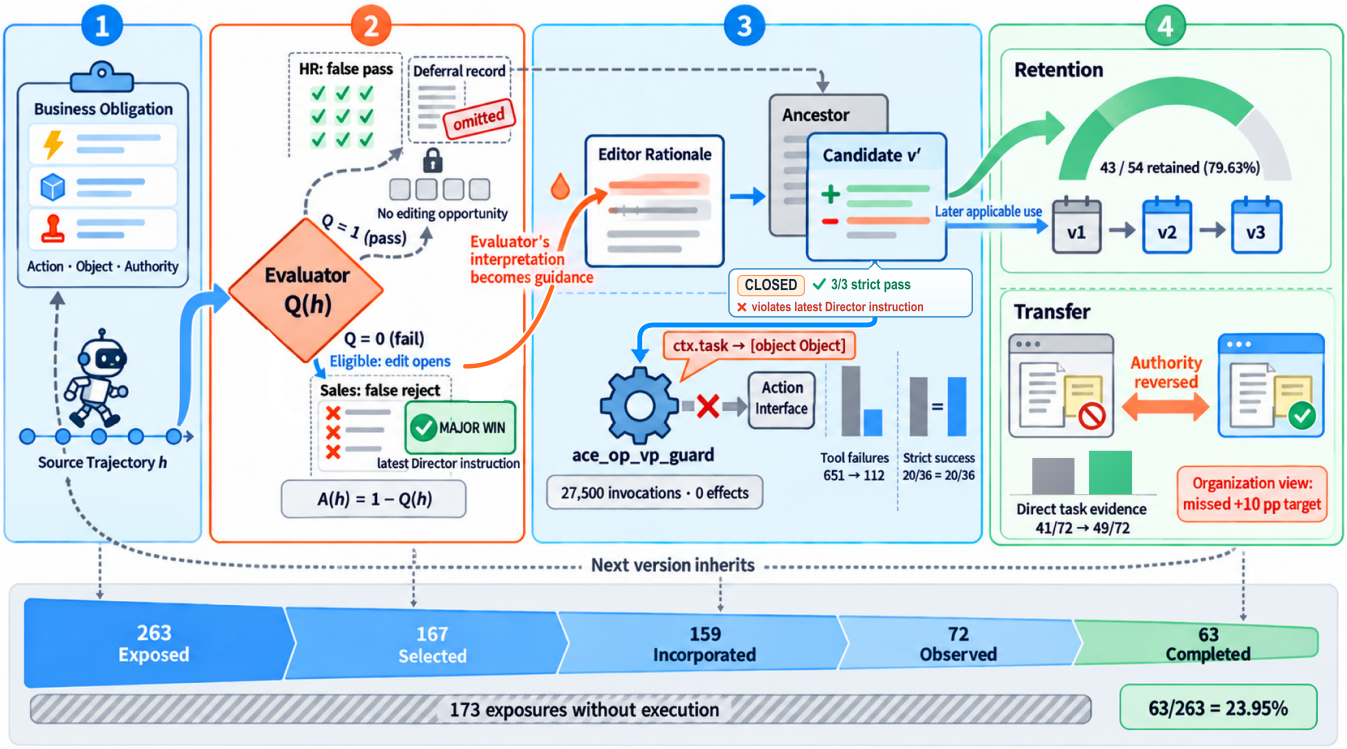


Figure 1. Evaluation shapes the construction of inherited behavior. (1) A trajectory expresses a business obligation. (2) Strict scoring controls editing: HR's false pass closes the repair opportunity, while the failed sales assertions supply the editor's guidance. (3) Candidates inherit that interpretation. All three legal sales edits produce `CLOSED`, pass strictly, and violate the latest Director instruction. Runtime effects establish whether inherited components act. (4) Later applicable tasks test retention and transfer. The lower path counts historical requirement exposures; its execution gaps are tracked separately.

request and saved rationale identify which obligation becomes an editing objective. Distinct exposures can concern the same obligation. For textual units, the model input establishes availability; behavioral evidence establishes what the execution does. For executable units, recorded branch effects establish an intervention. This preserves the distinction between content reaching the runtime and a business obligation being completed.

2.2 *The historical construction path*

Across 65 edits, we follow 263 feedback exposures through the editor's rationale and into subsequent business behavior. The editor selects 167 exposures in its rationales and fully incorporates 159 of those. Readable successor executions cover 72, and 63 reach completion at least once (Figure 2). Each completion concerns the obligation that motivated the edit, with its object and authority preserved. The resulting $63/263 = 23.95\%$ connects feedback to observed business completion. Another 20 complete incorporations fall outside the rationale-selected path, bringing the total to 179. For 173 exposures, the corresponding candidate receives zero executions, leaving the path open at deployment.

Each stage determines what the agent can carry forward. In the HR source, the omitted deferral must reach the editor as a failure before it can become a repair objective. An inherited guard needs usable task information and a matching runtime event to change an action. A later task introduces its own object and authority: a procedure learned for one announcement can prescribe the wrong action under a new instruction. The obligation gives us a common reference for tracing these changes from the original failure to the later action.

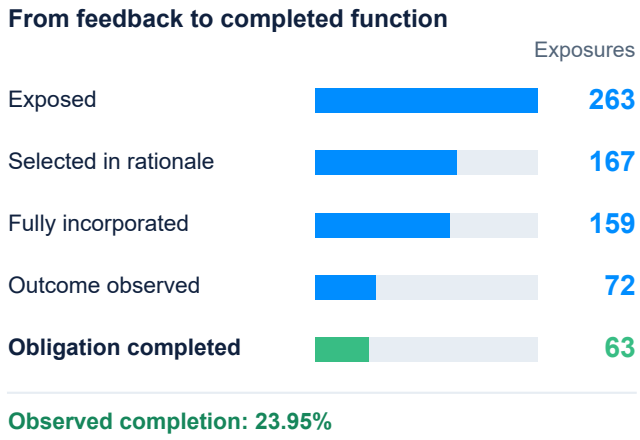


Figure 2. Nested requirement exposure sets across 65 edits. Completion concerns the exposed obligation in observed successors. Another 20 complete incorporations lie outside the rationale-selected path. The 173 exposures without candidate execution are tracked separately from these stage counts.

3 System and evidence

3.1 *Two decisions through which evaluation controls learning*

GNOSIS stores textual units and executable components in versioned collections. A working collection retains edits, while a released collection advances through independent comparisons. Units specify

runtime hooks and activation scope. Text can enter a model request; executable components can change arguments or observations and issue the task's original tool calls. The evaluator and permissions remain outside the agent's control.

The historical release rule requires observable improvement and zero task regression on six selection tasks, with two executions per task. A source must first succeed before a mature version comparison begins. In the frozen follow-up, strict failure on the source first run opens four editing slots, while strict success deploys the ancestor in all four. Every legal follow-up candidate is frozen before source retesting and transfer evaluation; source retests occur after deployment has been decided.

The eligible historical population contains 2,006 executions after the 7 October 2026 exclusion cutoff. The recovered graph has 130 vertices and 148 parent to child links across 211 version events. Proposal records include inspection actions as well as edits. Historical comparisons retain their batch conditions and task identities. The initial frozen evaluation, the subsequent interface studies, and the timing intervention are subsets of this historical population.

3.2 Frozen intervention and business completion

The follow-up uses one source from each of finance, HR, marketing, operations, sales, and support. Each source has four structurally related transfer tasks checked against all recorded prior exposure. All four editing conditions begin with the same ancestor, containing 13 textual units and one executable component. F0 supplies the complete source trajectory and assertion feedback. F1 retains those facts and adds a view organized by requirement, object, and action consequence. Its quotations resolve to the supplied source. R0 permits ordinary textual content. R1 requires nonempty `applies_when`, `avoids`, and `guidance` fields in the JSON content of changed units; unchanged ancestor units persist verbatim. This treatment adapts the boundary representation of BASM [6].

The common limits are 24 units and 80,000 serialized characters; the ancestor occupies 65,524 characters. Format failure permits one retry and then retains the ancestor. All failures and untriggered slots remain in the workflow outcome. Seed 20261014 fixes the editing order and interleaves transfer evaluation within source blocks. Each of the four edit conditions, the ancestor, and the carry condition runs four tasks three times per block: 432 executions. Six source first runs and 24 retests give 462 in total. The protocol was sealed on 14 October 2026.

The follow-up model is `deepseek-flash`, accessed through DeepSeek's official API. Requests enable thinking with high reasoning effort and temperature 1. Each response permits up to 393,216 output tokens. AutomationBench [9] is fixed at commit `4a8e106`, with its API toolset and at most 50 agent steps. Each run resets the task world and component state.

Task success requires all applicable obligations and normal termination, with execution errors counted as failures. Complete review defines obligations once per task and checks every run. A known failure fixes a failure label; remaining cases with unresolved obligations are unknown. Confirmed success uses all scheduled runs as its denominator. We retain original strict scoring and frozen assertion-indexed judgments alongside business review.

All 462 follow-up trajectories are replayed in the fixed official environment. Initial contracts, assertion outcomes, and strict success labels agree in $462/462$. The 63 historical executions with component effects also agree in $63/63$. Entity correspondence and runtime date differences remain separate from score agreement. An independent Codex agent checks 3,331 added obligation instances and reads 54 executions requiring semantic interpretation; 18 disagreements concern one quotation authority choice and alter one task label. The researcher reviews the final evidence. The supplement records provenance and all alternative authorization readings.

4 Evaluation defines the reachable repairs

4.1 A missing obligation becomes a missing learning opportunity

Let h be a source trajectory, $Q(h)$ its binary evaluator score, and $B(h)$ its complete business outcome. In the follow-up, edit eligibility is $A(h) = 1 - Q(h)$. For current version v and legal update operator U, the deployed version is

$$v' = \begin{cases} v, & Q(h) = 1, \\ U(v, h), & Q(h) = 0 \text{ and generation succeeds}, \\ v, & \text{generation fails}. \end{cases} \tag{1}$$

A false pass therefore changes the data generating process of learning. The omitted business failure receives no editing opportunity under this rule. Reformatting evidence downstream of A leaves that decision intact.

In the HR source, the latest eligibility email requires an Airtable deferral for the employee on sabbatical and leaves courses unassigned. The first run creates four active employee records and omits that deferral. All nine assertions pass. Equation (1) consequently closes all four editing slots, which deploy the ancestor. Every source retest repeats the business omission and passes the assertions. The unchanged content and the code path establish why this failure persists across the five observed executions.

The defect also gives the required correction a negative signal. Strict assertion 5 forbids `create_record` for the employee targeted by the deferral. Creating the required record through that operation would fail the assertion. The source pass therefore suppresses an attempt, and the conflicting assertion penalizes the corrective action. Proposition 1 states the conditions under which this conflict restricts the functional variants available to the learner.

Proposition 1 (Evaluator exclusion of a required correction) *Suppose business completion requires action D, evaluator success requires its absence, and a release rule preserves evaluator success on that task. From an evaluator-passing incumbent, every evaluated correction that performs D on the protected task is excluded. Under Equation* (1)*, an evaluator-passing source also produces zero edit attempts for that source, regardless of the available editing budget.*

To see this, write $\mathcal{B}_D$ for trajectories satisfying the required action and $\mathcal{Q}$ for evaluator-passing trajectories. The hypotheses give $\mathcal{B}_D \subseteq \{D = 1\}$ and $\mathcal{Q} \subseteq \{D = 0\}$, hence $\mathcal{B}_D \cap \mathcal{Q} = \varnothing$. Preserving evaluator success excludes every corrective trajectory. The source statement follows directly from $A = 0$. Removing the contradictory assertion removes this exclusion; changing the source eligibility rule removes the zero-attempt restriction. The HR execution instantiates the observed source restriction and the action conflict. Its four alternative edited lineages were never run.

4.2 Feedback can teach the evaluator's error

The sales source supplies the converse case. Under the main authorization reading, the latest Director instruction requires `MAJOR WIN`.

Table 1. Evaluation changes what is learned. Each entry is strict / complete business success. Sales uses the latest Director interpretation. Retests deploy the version recorded in the last column.

Source	Execution	Strict / business	Deployment
HR	First run	$1/0$	Ancestor
	Four retests	$1/0$ in all four	Ancestor
Sales	First run	$0/1$	Ancestor
	Three legal edits	$1/0$ in all three	Candidates
	Failed generation	$0/1$	Ancestor

The first run follows it and completes the business task. The original assertions demand `CLOSED` and prohibit `WIN:` and `MAJOR WIN:`, so editing begins. The editor's saved rationale explicitly cites the failed assertions, reclassifies the Director instruction as an unadopted suggestion, and elevates the older observed format to authority.

All three legal candidates rewrite that decision rule. F0R0 also installs a guard to normalize the announcement prefix. On source retest, all three strictly pass and violate the latest Director instruction. The fourth slot fails generation, retains the ancestor, and satisfies the business task while failing the assertions (Table 1). The edited candidates inherit the evaluator's interpretation through the editor's rationale and the resulting announcement rule.

The sales judgment follows the latest Director instruction. Under the older VP standard, `CLOSED` is correct. Sensitivity analysis evaluates all eight combinations of sales, quotation, and logging authority; the HR eligibility decision and the direction of the sales edits remain the same across these readings.

Across the 462 follow-up executions, complete review finds 262 confirmed successes, 179 failures, and 21 unknown outcomes. There are 102 strict passes with confirmed business failure and 38 strict rejections with confirmed business success. All 21 unknown runs strictly pass and remain outside those 102 errors. The source cases show how these disagreements affect learning: the score controls whether editing begins, and its assertions can become future guidance.

4.3 A saturated evaluator stops acceptance before money runs out

The earlier release experiment exposes a second restriction on reachable improvement. Both repair groups use a USD 8.5 allowance. The global validation group spends USD 3.028831 and GNOSIS USD 4.310552 before their released selection scores reach $12/12$. Their final comparisons give maximal candidate and reference scores and abstain. The remaining allowances are USD 5.471169 and USD 4.189448, respectively. GNOSIS's frozen held-out score is $101/150$: saturation of the 12 selection observations coexists with 49 failures in held-out execution.

Once every score used by a strict improvement rule reaches its maximum on a fixed panel, the rule has exhausted its admissible increases. Cheaper validation leaves this ceiling intact. Further accepted improvement requires feedback that can distinguish candidates at the remaining business failures.

5 Usable experience depends on runtime and task authority

After construction, a candidate must reach execution and satisfy an applicable obligation. The runtime studies follow its validation dependencies and component effects through to application state.

5.1 Validation locality is a property of information flow

Selective regression testing depends on the consumers of a change [2]. For changed agent units Δ, let $\mathcal{I}(u)$ contain historical executions whose runtime inputs or actions can depend on unit u. The validation set is

$$\mathcal{V}(\Delta) = \bigcup_{u \in \Delta} \mathcal{I}(u). \tag{2}$$

Globally exposed changes make every task a consumer. Declaring narrow intent leaves this dependency intact whenever the runtime still exposes the changed content to the entire panel.

All 48 historical GNOSIS candidate validation plans rerun the complete panel and reuse zero executions. Three independently learned source repairs in the locality study all remain global and cover $36/36$ audit runs each. A subsequent scope interface comparison produces 19 legal candidates, every one containing global content; its eight mature candidates all require $12/12$ audit runs. Changing the declaration format leaves the runtime dependency broad.

The 240-execution timing intervention tests the proposed explanation that local guidance arrived too late. It fixes eight repairs and compares global early, local early, local delayed, contract early with body delayed, and removed guidance. Local early is noninferior to global early: $+2.08$ pp, with interval $[-4.17, 8.33]$ pp at the prespecified ten point margin. The planning versus execution-detail interaction for local early minus delayed is -4.17 pp, $p = 1.0$, interval $[-25.00, 12.50]$ pp. The prespecified analysis finds no reproducible timing pattern. Host dependency sets still cover every task in each of the eight audit pools, and routing plus prediction makes selective validation more expensive.

Early local access preserves utility under a fixed route. Every audit task still depends on the changed content, so routing saves zero validation runs. Selective validation requires some consumers to remain independent of the whole change. Equation (2) locates the observed burden in this information flow.

5.2 Executable inheritance can preserve an inactive procedure

Across history, 27 executable units enter 447 executions and produce 196 effects in 63 distinct executions; components themselves issue 27 tool calls. Of the 13 effect-bearing runs covered by complete business review, six succeed and seven fail. The runtime supports interventions, whose business consequences require a further check.

The inherited `ace_op_vp_guard` receives 27,500 invocations across history and follow-up, with zero effects and zero component tool calls. The count separates into 720 task starts, 26,745 URLs outside its target branch, and 35 target URLs. Of the target cases, 23 obtain no usable prefix, ten already have the correct prefix, and two have invalid JSON bodies. The host passes `ctx.task` as an object, while the component reads `String(ctx.task)` and obtains `[object Object]`. The intended instruction is lost at the interface. Isolated replay of the original code in 45 saved real contexts reproduces zero argument changes and zero calls; all ten task starts reproduce that object conversion.

5.3 Shared interfaces leave business decisions task dependent

The subsequent frozen comparison on 18 new tasks makes the distinction between operational and business transfer visible. GNOSIS reduces failed tool calls from 651 to 112 and average actions from

Table 2. Operational improvement and task completion on the same 18 tasks, two runs each. Failed calls are totals; actions are means; expenditure covers the 36 executions in CNY.

Agent	Failed calls	Actions	Success	Cost
Fixed	651	75.14	20/36	7.8513
Global	165	45.47	25/36	3.9763
GNOSIS	112	39.31	20/36	3.2733

75.14 to 39.31, while strict success stays at 20/36 (Table 2). Its execution expenditure falls from CNY 7.8513 to CNY 3.2733. Across 377 matched parent and candidate comparisons, 217 reduce tool failures; 159 of those do so without increasing strict success. These comparisons retain their shared execution identities.

Business completion requires the right operation on the right object under the current authority. Successful interface use addresses the first of these conditions. The inherited rule must still identify which document controls the task and which entities or exceptions it covers. The target task can reverse the source decision while using the same application (Table 3). All 24 transfer tasks receive a reading against candidate contents and their own authorized materials.

Several candidates retain source literals and authority decisions. Table 3 identifies target conditions under which those rules require revision. The aggregate experiment measures the full workflow across all tasks, including sources that retain the ancestor.

6 Transfer tests the function constructed from source feedback

6.1 The organization intervention changes a restricted part of learning

Detailed assertion feedback enters edit rationales in 154/187 exposures (82.35%), compared with 7/70 task summaries (10.00%). Twelve of 17 repeatedly exposed requirement groups are never selected. This motivates the organization intervention: F0 and F1 receive the same complete facts, and F1 adds an organized view after eligibility. Both conditions retain the eligibility rule and the destination task's authority.

Twelve of the 24 slots never enter editing because HR, marketing, and operations strictly pass. The other 12 slots produce nine legal candidates and three terminal format failures, all under R1. Six R0 candidates change 51 units; three legal R1 candidates each change one. All slots retain their observed deployment in Table 4. The treatment includes its generation burden and the updates it actually produces.

For source block b, the primary difference averages four transfer tasks and three repetitions, then averages equally across the six blocks:

$$d_b = \frac{1}{12}\sum_{t=1}^{4}\sum_{r=1}^{3}\left(y_{btr,\mathrm{F1R0}} - y_{btr,\mathrm{F0R0}}\right),$$
$$\widehat{\Delta} = \frac{1}{6}\sum_{b} d_b. \tag{3}$$

The frozen test enumerates 64 sign assignments, with a 95% common shift inversion interval on a 0.0001 grid. Inference assumes exchangeable block signs and a common shift. The prespecified practical target is $+10$ pp.

Under frozen judgments, F1R0 minus F0R0 is -9.72 pp, $p = 0.25$, interval $[-33.33, 8.33]$ pp. Its upper endpoint is below the practical target, so organization fails the prespecified decision. Complete business review gives -1.39 pp, $p = 1.0$, interval

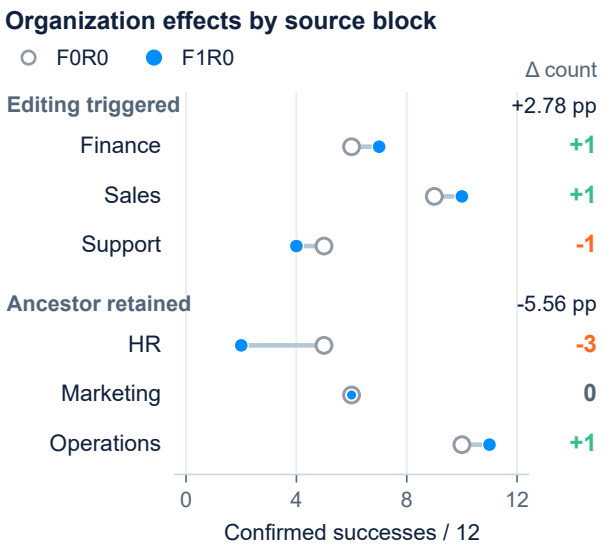


Figure 3. Business success under F0R0 and F1R0, grouped by actual editing eligibility. Each source block has 12 runs per condition. The right column gives count differences; group headings report mean differences in percentage points. Ancestor-retaining blocks use identical artifacts and inputs.

$[-25.00, 8.33]$ pp. The alternative quotation authority gives zero difference and interval $[-25.00, 16.66]$ pp. Assigning unknown obligations to minimize or maximize the contrast bounds the observed difference by $[-6.94, 4.17]$ pp; the maximizing assignment has interval $[-8.33, 16.66]$ pp.

The three sources that trigger editing average a $+2.78$ pp business contrast; those retaining the ancestor average -5.56 pp (Figure 3). The latter have identical content and inputs across conditions, so their differences arise in execution. Sixty groups with the same version, task, initial state, and input cover 360 paid runs; 328 of 1,548 within-group pairs change frozen labels (21.19%). These pairs share executions. The R1 simple increment and interaction have Holm adjusted $p = 1.0$, and no block supplies four legal edited candidates. The negative result concerns the full implemented workflow, including unchanged deployments and failed generation.

6.2 Direct task evidence supports correction

Carry receives its preceding ancestor trajectory with actions, tool returns, and assertion feedback, then restarts the original task world. Fresh retry uses the three ancestor runs in a fixed cyclic pairing per task. The 72 pairs therefore contain 72 distinct ancestor runs, each used once as a first attempt and once as another pair's retry. Aggregation preserves this sharing.

Carry raises frozen success from 55/72 to 72/72. Nine extra successes follow a failed first run and eight follow a passing first run. Complete business success increases from 41/72 to 49/72. The complete two-attempt cost increases from CNY 13.881212 to CNY 14.497128, or CNY 0.008554 per pair. Cost per confirmed success falls from CNY 0.338566 to CNY 0.295860. The prespecified lower total cost criterion fails; the observed benefit is eight additional confirmed successes for CNY 0.615916 more.

Direct evidence improves the fixed model's next attempt on the same task (Figure 4). Transfer additionally requires an inherited decision to remain valid when objects and authority change. F1 organizes

Table 3. Authority and identity conditions change across tasks. Source rules occur in edited candidate content; target requirements come from the frozen task materials.

Requirement	Inherited source decision	Authorized target decision
Sales authority	Ignore account notes that redirect a deal announcement; retain the established announcement channel.	The DocuSign completion runbook explicitly authorizes account notes to determine credit holds and partner billing.
Support identity	Group duplicates by exact email; records from different email addresses remain separate.	The cross-platform deduplication task supplies an alias table identifying different addresses as one customer and requires reciprocal internal notes.

Table 4. Full workflow outcomes on 24 transfer tasks, 72 runs per condition. Percentages use the complete denominator, including unknown outcomes. Every editing slot retains its actual deployment.

	Source deployment slots		Original and frozen evaluation		Complete business review	
Condition	Legal edits	Ancestor	Strict success	Frozen success	Confirmed success	Unknown
F0R0	3	3	56 (77.8)	58 (80.6)	41 (56.9)	4
F1R0	3	3	49 (68.1)	51 (70.8)	40 (55.6)	4
F0R1	1	5	51 (70.8)	53 (73.6)	43 (59.7)	3
F1R1	2	4	45 (62.5)	47 (65.3)	36 (50.0)	2
Ancestor / fresh retry	—	6	51 (70.8)	55 (76.4)	41 (56.9)	4
Carry	—	6	72 (100.0)	72 (100.0)	49 (68.1)	4



Figure 4. Persistent transfer and direct task evidence. (a) Confirmed business success uses all 72 scheduled runs per condition. F1R0 adds organized source feedback; Carry receives the preceding trajectory and evaluator feedback for the same task. (b) Actual costs of 72 two-attempt pairs, including the first attempt. Carry buys eight additional confirmed successes for CNY 0.615916 more and lowers cost per confirmed success.

the source facts; execution must still establish whether the resulting guidance applies at the destination.

7 Acquisition and retention of business function

Across all successors of the candidates traced in Section 2, 117 requirement and execution observations contain 92 passes, 12 failures, 12 summaries without an identifiable requirement, and one unloaded component. Twenty requirement successes occur inside whole task failures. An agent can satisfy one obligation while failing another required for task success. Tracing individual requirements preserves these partial achievements for later comparison.

Retention begins at a verified functional success. Among 83 original assertion sequences, 54 have a subsequent applicable opportunity and 43 show no later failure, a conditional retention rate of 79.63%. The other 29 lack follow-up. Expanded tracing of 196 event sequences gives 111 retained, 19 later failed, 60 without a successor, and six without a verified starting success. These observations establish that some completed functions persist across versions. Their opportunity counts distinguish sustained function from a version that simply continues to contain an artifact.

The protocol follows each obligation through the decisions needed to establish accumulated function. Task materials supply its authority; the score controls eligibility. The rationale and version difference show what the editor constructs. Successor inputs and component effects locate its expression, and application state establishes completion. Later applicable opportunities test retention; new tasks test the validity of transferred decisions. Failed generation and unchanged deployments retain their expenditure and outcomes.

8 Related work

Reflexion [10], ACE [16], and Voyager [12] establish verbal and executable carriers of experience. EASE [13] studies skill curation conditioned on downstream executor behavior. BASM [6] represents applicability and failure conditions within retrieval and action checking. R1 changes how edited units are represented; its outcomes include failed generation and unchanged ancestor deployments.

Agent Plasticity [11] studies inheritance and acquisition efficiency, including failures despite relevant artifacts. EvoPathBench [5] measures acquisition and retention alongside the gap between candidate improvements and selected updates. SkillScriptBench [7] follows documentation and script repair through preservation. We extend these process questions to business obligation identities and the control decisions that determine whether a repair is attempted.

Execution Consistency [14] follows information through use and obligation completion. GAUGE [1] studies evaluation validity and candidate rankings; VeriHarness [15] checks disputed claims and omissions against environmental evidence. SilentProbe [4] studies failures despite apparently valid API behavior. We trace the evaluator's judgment into a persistent change, retaining obligations whose learning opportunity closes before an edit exists. Our validation argument draws on regression test selection [2]: the consumers of edited content determine the validation set.

9 Conclusion

Evaluation controls which business errors become learning objects and which interpretation enters persistent experience. In GNOSIS, the HR eligibility branch suppresses a needed correction. The sales edits inherit a scoring interpretation that reverses the latest Director instruction. A saturated selection panel also stops acceptance while money and held-out failures remain.

Following each obligation from feedback through runtime expression establishes where an inherited artifact supplies function. Later applicable opportunities test whether that function persists. Direct task evidence improves the next attempt, while the implemented organization treatment misses its transfer target.

References

[1] Umesh Bodhwani, Thanh Tran, and Kai Wei. GAUGE: When not to trust LLM-as-a-judge in user-simulated evaluation of task-oriented agents. arXiv:2609.12191v1, 2026.

[2] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov, 'Practical regression test selection with dynamic file dependencies', in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, pp. 211–222. ACM, (2015). https://doi.org/10.1145/2771783.2771784.

[3] Marc Kirschner and John Gerhart, 'Evolvability', *Proceedings of the National Academy of Sciences*, **95**(15), 8420–8427, (1998).

[4] Zongrong Li, Shengkun Ye, Feiyou Guo, and Zuoyou Dang. SilentProbe: Measuring silent failure in production APIs used as agent tools. arXiv:2609.00035v1, 2026.

[5] Hongqiang Lin, Chao Liu, Xiaofan Bai, Xuan Jin, Yuhong Li, Nenggan Zheng, and Xipeng Cao. Beyond endpoint performance: Process-level evaluation of self-evolving agents. arXiv:2609.24663v1, 2026.

[6] Zihan Lin, Zhenyu Chen, Jiawen Wei, Xiaohan Wang, Jie Cao, Jiajun Chai, Wei Lin, Guojun Yin, and Ran He. When not to imitate: Boundary-aware skill memory for reliable tool-use LLM agents. arXiv:2608.22339v1, 2026.

[7] Yuxuan Liu, Haoran Li, Yuhao Zhang, Jiahe Guo, Hongyu Luo, Wenbin Hu, Huihao Jing, Kawai Chung, Junle Chen, Changxuan Fan, Qing Zong, Lingyun Xie, and Yangqiu Song. SkillScriptBench: Benchmarking self-evolution of executable agent skill packages beyond Markdown. arXiv:2610.04008v1, 2026.

[8] Erwin Schrödinger, *What Is Life? The Physical Aspect of the Living Cell*, Cambridge University Press, 1944.

[9] Daniel Shepard and Robin Salimans. AutomationBench. arXiv:2604.18934, 2026.

[10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. arXiv:2303.11366, 2023.

[11] Harman Singh, Anton Bakhtin, Rulin Shao, Gabriel Synnaeve, Ilia Kulikov, Rob Fergus, Sanjeev Arora, Kurt Keutzer, Jason Weston, Anuj Mahajan, and Anirudh Goyal. Agent plasticity: Measuring self-improvement through experience. arXiv:2610.08902v1, 2026.

[12] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv:2305.16291, 2023.

[13] Zhen Xiong and Qiaoyu Tan. EASE: Behavior-adaptive skill curation for self-evolving agents. arXiv:2609.36746v1, 2026.

[14] Zhe Yu, Zixuan Wang, Peidong Wang, Hehai Lin, Ruochen Zhao, and Chengwei Qin. Beyond corrected memory: Execution consistency in multi-agent systems. arXiv:2610.08101v1, 2026.

[15] Caiqi Zhang, Rujun Han, Zifeng Wang, Zoey CuiZhu, Nigel Collier, Tomas Pfister, and Chen-Yu Lee. VeriHarness: Scaling agentic verification for long-horizon tasks. arXiv:2610.00972v1, 2026.

[16] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun, 'Agentic context engineering: Evolving contexts for self-improving language models', in *International Conference on Learning Representations*, (2026).